

Hashing and More Design Problems

5/19/25

Administrivia

- HW 9 (sorting, heaps, fork-join) due Thursday night

Recall: Lab 8 (counting primes with fork-join)

- Initial (serial) version: 227ms
- With tasks:
 - 1 -> 230ms
 - 2 -> 155ms
 - 3 -> 120ms
 - 4 -> 112ms
- Best threshold: 400,000 (4 tasks); gives time around 110

Back to Set and Map

- Need to quickly find a key (and associated value)
- Main operations are find, insert, delete

A different approach: Hashing

0	1	2	3	4	5	6	7

- Based on an array
 - Use hash function to convert key to integer
 - Use % to convert it to an array index
- Closed addressing:
 - Each entry is a list of keys mapping to that cell

Where does 12 go in the following hash table?

(Assuming identity hash function)

0	1	2	3	4	5	6	7

A. 0

B. 2

C. 4

D. 6

E. None of the above

Where does 12 go in the following hash table?

(Assuming identity hash function)

0	1	2	3	4	5	6	7

A. 0

B. 2

C. 4

D. 6

E. None of the above

Alternate: Open addressing

0	1	2	3	4	5	6	7

- Only one key can be in each array cell
- When a second key tries to join, find a different place
 - Linear probing
 - Quadratic probing
 - Double hashing

Tricky point: Removing a key

- Can't just remove it or other keys might disappear from the table
- Instead, replace with a “bridge” that has no key but represents a full cell

Consider an initially empty hash table of size 7 implemented with open addressing and linear probing. Draw a diagram showing its contents after each operation in the following sequence (using the value of each number as the value of its hash function):
add(10), add(3), add(4), add(17), delete(4)